

UNIVERSIDAD SAN IGNACIO DE LOYOLA

FACULTAD DE INGENIERÍA

Producto Acreditable - Avance 1

Profesor: Castañeda Alban, Jorge Eduardo

Integrantes:

Cardenas Tantalean, Jhair Alexis

García Serna, Jackeline Edith

Velasquez Silva, Andrea Estephany

Salazar Goncalves, Dominith Leoncio

Lima - Perú

2026 – II

Índice

1. Análisis y modelado del problema..... 3
2. Modelo geométrico y definición del grafo de navegación..... 4
3. Catálogo de restricciones, datasets y métricas de evaluación..... 5
3.1 Categorías de prueba y límites competitivos..... 5
3.2 Formato de archivos de entrada y salida..... 5
3.3 Autonomía de batería y modelo de consumo..... 6
3.4 Métricas de evaluación y sistema de puntuación..... 7
3.5 Estrategia de generación de datasets..... 8
4. Pseudocódigo y análisis de complejidad de los algoritmos seleccionados..... 8
5. Lista formal de requerimientos, arquitectura y plan de pruebas..... 9
5.1 Requerimientos funcionales..... 9
5.2 Requerimientos no funcionales..... 10
5.3 Arquitectura modular en C++ (separación en capas)..... 11
5.4 Plan de pruebas..... 13
5.5 Matriz de trazabilidad..... 15

1. Análisis y modelado del problema

- ***Descripción del caso de estudio de GeoDrone Logistics***

GeoDrone Logistics es una empresa de distribución urbana que emplea drones autónomos para realizar entregas de última milla. Cada jornada parte de un centro logístico y debe atender múltiples clientes ubicados en diferentes coordenadas de la ciudad. La operación está condicionada por zonas restringidas de vuelo, edificios y obstáculos representados mediante polígonos, autonomía limitada de batería, estaciones de recarga y ventanas de tiempo.

GeoDrone Logistics debe planificar misiones de entrega en un entorno urbano donde la trayectoria geométricamente más corta no siempre es válida debido a obstáculos y zonas de exclusión. Además, cuando una misión incluye numerosos clientes, determinar el orden óptimo de visita se convierte en un problema combinatorio de alta complejidad (una variante del problema del viajante de comercio, TSP, con restricciones). El reto consiste en determinar qué combinación de técnicas algorítmicas permite obtener rutas válidas y de buena calidad dentro de límites estrictos de tiempo, memoria y autonomía, distinguiendo cuándo es viable una solución exacta y cuándo se requiere una estrategia heurística o aproximada.

El sistema representará el escenario mediante entidades geométricas y un grafo ponderado G = (V, E), donde los vértices corresponden a la base, los clientes, las estaciones de recarga y los puntos relevantes de los obstáculos; las aristas representan desplazamientos válidos y sus pesos expresan distancia, tiempo o consumo estimado de batería. Sobre este grafo se calculan caminos mínimos y, a partir de ellos, se optimiza la secuencia de visitas.

- ***Identificación de entidades clave:***
    - Base: Es el punto desde donde parte el dron
    - Clientes: Son las personas a quienes se les debe entregar un pedido
    - Estaciones: Son lugares donde un dron puede recargar su batería
    - Obstáculos poligonales: Son los edificios o espacios por donde un dron no puede cruzar.

- ***Criterios operativos y factibles de vuelo:***

- Cada jornada parte de un centro logístico y debe atender múltiples clientes ubicados en diferentes coordenadas de la ciudad.
- Existen zonas restringidas de vuelo, edificios y obstáculos representados mediante polígonos.
- El dron tiene autonomía limitada de batería
- Existen estaciones de recarga

2. Modelo geométrico y definición del grafo de navegación

- ***Descripción y representación Matemáticas:*** (producto cruz, orientación, intersección, punto en polígono, distancia)

  - **Punto (x,y):** Un punto es una ubicación en el espacio, en coordenadas cartesianas se escribe como par ordenado (A, B) o una terna en 3D. Permite realizar operaciones vectoriales básicas, distancia euclidiana y Manhattan.
  - **Segmento (p1, p2):** Un segmento de línea es una sección finita de una línea recta con dos puntos finales y una longitud fija. Dados dos puntos distintos A y B En una línea, el segmento de línea AB⁻A B Es el conjunto de todos los puntos en la línea que están entre AA y BB, junto con los puntos AA y BB Ellos mismos. Su longitud, denotada ABAB, es la distancia entre los dos puntos finales.
  - **Polígono (vértices):** Un polígono es la región de un plano delimitada por una colección finita de segmentos de recta que forman una curva cerrada simple.
  - **Orientación / Producto Cruz:** Determina si tres puntos forman un giro horario, antihorario o son colineales. Es decir al ser un punto A, estar frente a un punto B y tener un punto C saber si esta esta a la derecha, a la izquierda o alineado con el punto A.
  - **Intersección de segmentos:** Detección robusta de colisión considerando casos estándar y degenerados (puntos colineales, contacto en vértices y traslape).
  - **Punto en polígono (Point in polygon):** Algoritmo de Ray Casting o Winding Number, para identificar si un punto está dentro o fuera de un polígono.
  - **Validación de Línea de vista (Line of Sight):** Función para evaluar si un trayecto entre dos puntos atraviesa el interior de algún obstáculo.

- ***Reglas de validez:*** cuándo un vuelo es válido (por ejemplo, qué pasa si solo toca el borde de un edificio).

- ***Definición formal del grafo de visibilidad:*** G = (V, E)

3. Catálogo de restricciones, datasets y métricas de evaluación

3.1 Categorías de prueba y límites competitivos

La Tabla 1 define las cuatro categorías de datasets con sus rangos de tamaño y los límites de tiempo de ejecución y memoria. La guía del curso exige límites explícitos por categoría pero no fija sus valores, por lo que los números siguientes son la propuesta del equipo, calculados a partir de la complejidad de los algoritmos seleccionados (sección 4) y ajustables tras medir el prototipo del Entregable 2.

Tabla 1. Categorías de datasets, rangos de tamaño y límites competitivos (propuesta del equipo).

Categoría	Clientes (k)	Estaciones	Polígonos	Vértices totales	Tiempo máx.	Memoria máx.	Estrategia principal
Small	5 – 12	1 – 3	1 – 5	≤ 30	1 s	256 MB	Exacta (B&B)
Medium	13 – 100	3 – 10	5 – 30	≤ 150	2 s	256 MB	Heurística + 2-opt
Large	101 – 300	10 – 50	30 – 100	≤ 400	5 s	512 MB	ILS / multi-inicio
Extreme	301 – 1000	50 – 100	100 – 300	≤ 1500	10 s	1024 MB	Heurística escalable

3.2 Formato de archivos de entrada y salida

Los escenarios y los resultados se intercambian en archivos de texto plano (.txt), lo que facilita la lectura rápida con I/O de C++ y la comparación automática por el juez. Todos los números de la entrada son enteros, excepto los parámetros del dron, que pueden ser decimales. Los vértices de cada polígono se listan en sentido antihorario.

Formato de entrada.

```
SEED <semilla>   // semilla con la que se generó el caso
CATEGORY <Small|Medium|Large|Extreme>
<k> <m> <p>      // n° de clientes, estaciones, polígonos
```

```
A <autonomía> KAPPA <κ> V <v> Q <Q> TAU <τ> KH <κh>
BASE <x> <y>
CLIENT <id> <x> <y> <prio> <obligatorio 0|1> <e> <l> <s>
<demanda>   (k líneas)
STATION <id> <x> <y>
(m líneas)
POLYGON <id> <nv> //seguido de nv líneas "x y" (p bloques)
```

Ejemplo de entrada (escenario de la Figura 1, con A = 20):

```
SEED 20261
CATEGORY Small
2 1 1
A 20 KAPPA 1 V 1 Q 10 TAU 0 KH 0
BASE 1 4
CLIENT 1 11 4 2 1 0 1000 0 1
CLIENT 2 6 9 1 1 0 1000 0 1
STATION 1 9 0
POLYGON 1 4
4 2
8 2
8 6
4 6
```

Formato de salida. El solver imprime el estado, el costo, el tiempo, las recargas y la ruta como secuencia de identificadores: B (base), C# (cliente), S# (estación) y W#.# (vértice de obstáculo usado como punto de paso: polígono.vértice).

```
STATUS FEASIBLE          // FEASIBLE | INFEASIBLE
COST 27.605
TIME_MS 1
RECHARGES 1
SKIPPED 0                // clientes opcionales omitidos
ROUTE 6
B C2 C1 S1 W1.1 B
```

En este ejemplo la ruta óptima es B → C2 → C1 → S1 → V1 → B con costo 27,605: tras visitar C1 quedan 5,86 unidades de batería, que alcanzan para llegar a la estación S1 (4,47), recargar y regresar a la base (8,99). El orden alternativo B → C1 → C2 es infactible con A = 20.

3.3 Autonomía de batería y modelo de consumo

El consumo es lineal en la distancia, con un término adicional por el tiempo de servicio (el dron mantiene el vuelo estacionario mientras entrega). Para un tramo (u, v) y un servicio en v:

$$consumo(u, v) = \kappa \cdot d(u, v) + \kappa_h \cdot s_v$$

Los parámetros iniciales por categoría se muestran en la Tabla 2. La autonomía se fija de modo que el dron pueda recorrer aproximadamente entre una y dos veces la diagonal del

escenario, lo que obliga a usar estaciones de recarga en las categorías grandes. La recarga en estación o base es completa y tarda τ segundos.

Tabla 2. Parámetros iniciales del dron y del escenario por categoría (propuesta del equipo).

Categoría	Coordenadas	Autonomía A	Consumo κ	Velocidad v	Servicio s / κ_h / τ
Small	[0, 1000]	1500	1 u/m	10 m/s	30 s / 0,2 u/s / 60 s
Medium	[0, 5000]	3000	1 u/m	10 m/s	30 s / 0,2 u/s / 60 s
Large	[0, 10000]	4000	1 u/m	10 m/s	30 s / 0,2 u/s / 60 s
Extreme	[0, 20000]	5000	1 u/m	10 m/s	30 s / 0,2 u/s / 60 s

3.4 Métricas de evaluación y sistema de puntuación

Tabla 3. Métricas de evaluación de cada ejecución.

Métrica	Definición	Unidad / criterio
Factibilidad	La ruta cumple F1–F5 verificados por el juez de forma independiente al solver.	Binaria; una ruta infactible puntúa 0.
Costo total	C(R) = D(R) + penalización de opcionales omitidos.	Metros (menor es mejor).
Tiempo de ejecución	Tiempo de pared medido con <chrono> desde la lectura hasta la escritura de la solución.	Milisegundos; debe ser ≤ T_{max}.
Memoria	Pico de memoria residente del proceso.	MB; debe ser ≤ M_{max}.
Gap respecto a la mejor solución conocida	gap = (C − C*) / C* · 100, con C* el óptimo (Small, por B&B) o la mejor solución conocida (resto).	Porcentaje (menor es mejor).
Calidad	calidad = 100 · C* / C	Porcentaje (100 = igual a la referencia).

Sistema de puntuación tipo juez. Cada ejecución recibe un veredicto: **AC** (aceptado: factible y dentro de los límites), **WA** (respuesta incorrecta: infactible), **TLE** (excede el tiempo), **MLE** (excede la memoria) o **RE** (error de ejecución). El puntaje de un caso es 0 si el veredicto no es AC; en caso contrario es min(100, calidad). El puntaje de una categoría es

el promedio de sus casos y el puntaje final pondera las categorías (Small 10 %, Medium 20 %, Large 30 %, Extreme 40 %). En caso de empate, se desempata por menor tiempo total.

3.5 Estrategia de generación de datasets

- **Reproducibilidad:** cada caso se genera con std::mt19937_64 inicializado con una semilla registrada en el propio archivo (línea SEED); la semilla de cada caso es semilla_base + índice. La misma semilla debe producir exactamente el mismo archivo.
- **Casos aleatorios:** clientes y estaciones uniformes en el rectángulo de la categoría; polígonos convexos pequeños generados por muestreo con rechazo para que sus interiores sean disjuntos y no contengan nodos. El generador garantiza que cada cliente sea alcanzable (condición de la sección 2.5).

Casos adversariales. Se añaden escenarios diseñados para romper implementaciones ingenuas (Tabla 4).

Tabla 4. Casos adversariales del generador.

Caso	Descripción	Qué verifica
Colinealidad	Clientes y vértices alineados con aristas de polígonos; clientes sobre vértices o bordes.	Exactitud de los predicados geométricos.
Pasillos estrechos	Obstáculos separados por huecos mínimos, casi un laberinto.	Corrección del grafo de visibilidad y conectividad.
Cliente encerrado	Cliente rodeado por polígonos o fuera de alcance de toda estación.	Detección de infactibilidad sin colgarse.
Autonomía en el límite	A igual a la distancia mínima exacta que hace factible la misión.	Precisión numérica y búsqueda binaria.
Clusters lejanos	Grupos de clientes muy separados que engañan al vecino más cercano.	Calidad de la mejora local
Polígonos cóncavos grandes	Pocos polígonos con muchísimos vértices.	Peor caso del grafo de visibilidad y del test de validez.

4. Pseudocódigo y análisis de complejidad de los algoritmos seleccionados

- **Dijkstra:** Calcula el camino más corto entre pares de puntos en el grafo de visibilidad. Como las distancias y consumos son reales y no negativos. Dijkstra con

cola prioridad es el estándar óptimo para calcular trayectorias punto a punto esquivando polígonos. Su complejidad es de O((V+E) log V).

```
DIJKSTRA(G, s):
    // G = (V, E, w) con w(u,v) >= 0
    // s = vértice origen
    para cada v en V:
        dist[v] ← ∞
        padre[v] ← NULO
    dist[s] ← 0
    Q ← cola de prioridad mínima vacía
    insertar (0, s) en Q
    mientras Q no esté vacía:
        (d, u) ← extraer mínimo de Q
        si d > dist[u]:          // entrada obsoleta, se ignora
            continuar
        para cada (v, w(u,v)) en Adyacentes[u]:
            si dist[u] + w(u,v) < dist[v]:
                dist[v] ← dist[u] + w(u,v)
                padre[v] ← u
                insertar (dist[v], v) en Q
    retornar dist, padre
```

- **Floyd-Warshall:** Permite obtener la matriz completa de distancias mínimas entre los puntos clave (base, clientes, cargadores). Si el número de vértices de interés es moderado, simplifica la consulta de distancias para el ruteo posterior. Su complejidad es de O(V^3).

```
FLOYD_WARSHALL(G):
// G = (V, E, w), con V = {1, ..., N}
// w(i,j) puede ser ∞ si no hay arista directa
para i desde 1 hasta N:
    para j desde 1 hasta N:
```

```
        si i == j:
            dist[i][j] ← 0
            sig[i][j] ← j
        sino si existe arista (i,j):
            dist[i][j] ← w(i,j)
            sig[i][j] ← j
        sino:
            dist[i][j] ← ∞
            sig[i][j] ← NULO
para k desde 1 hasta N:              // vértice intermedio
    para i desde 1 hasta N:
        para j desde 1 hasta N:
            si dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] ← dist[i][k] + dist[k][j]
                sig[i][j] ← sig[i][k]
retornar dist, sig
```

- **Búsqueda por rangos:** Permitirá optimizar la construcción del grafo, permitirá podar candidatos de intersección con polígonos, así como identificar las estaciones de recarga o clientes que están dentro del radio de autonomía de batería. Su complejidad es de O(logN + K).

```
LIMITE_INFERIOR(A, x):          // primer índice con A[i] >= x
    lo ← 0
    hi ← |A|
    mientras lo < hi:
        mid ← (lo + hi) / 2
        si A[mid] < x:
            lo ← mid + 1
        sino:
            hi ← mid
```

```
    retornar lo
LIMITE_SUPERIOR(A, x):          // primer índice con A[i] > x
    lo ← 0
    hi ← |A|
    mientras lo < hi:
        mid ← (lo + hi) / 2
        si A[mid] <= x:
            lo ← mid + 1
        sino:
            hi ← mid
    retornar lo
BUSQUEDA_POR_RANGO(A, a, b):
    // A ordenado ascendentemente
    inicio ← LIMITE_INFERIOR(A, a)
    fin ← LIMITE_SUPERIOR(A, b)
    retornar A[inicio .. fin - 1]
```

- **DFS:** Permitirá identificar puntos de articulación (estaciones críticas) y puentes (corredores aéreos sin redundancia).

```
        DFS(G, s):
    para cada v en V:
        visitado[v] ← falso
        padre[v] ← NULO
    orden ← lista vacía
    DFS_VISITAR(G, s)
    retornar orden, padre

DFS_VISITAR(G, u):
    visitado[u] ← verdadero
    agregar u a orden               // preorden
    para cada v en Adyacentes[u]:
```

```
    si no visitado[v]:
        padre[v] ← u
        DFS_VISITAR(G, v)
```

- **Held-Karp:** Este algoritmo permitirá resolver el TSP de visitas a clientes de forma exacta. Su complejidad es de O($n^2 2^n$).

```
HELD_KARP(M, n):
// M = matriz de costos (n+1) x (n+1), vértice 0 = depósito
// clientes = 1..n
FULL ← (1 << n) - 1
para cada S desde 0 hasta FULL:
    para j desde 1 hasta n:
        dp[S][j] ← ∞
        padre[S][j] ← NULO
// Caso base: subconjuntos de un solo cliente
para j desde 1 hasta n:
    dp[1 << (j-1)][j] ← M[0][j]
// Transición
para S desde 1 hasta FULL:                  // de menor a mayor máscara
    para j desde 1 hasta n:
        si bit (j-1) de S no está activo:
            continuar
        R ← S sin el bit (j-1)              // S \ {j}
        si R == 0:
            continuar                       // caso base
        para k desde 1 hasta n:
            si bit (k-1) de R está activo:
                costo ← dp[R][k] + M[k][j]
                si costo < dp[S][j]:
                    dp[S][j] ← costo
                    padre[S][j] ← k
```

```
// Cierre del ciclo: volver al depósito
optimo ← ∞
ultimo ← NULO
para j desde 1 hasta n:
    costo ← dp[FULL][j] + M[j][0]
    si costo < optimo:
        optimo ← costo
        ultimo ← j
// Reconstrucción de la ruta
ruta ← lista vacía
S ← FULL
j ← ultimo
mientras j ≠ NULO:
    agregar j a ruta
    k ← padre[S][j]
    S ← S sin el bit (j-1)
    j ← k
invertir ruta
ruta ← [0] + ruta + [0]
retornar optimo, ruta
```

- **Branch & Bound:** Este algoritmo permitirá podar ramas del árbol de búsqueda cuando la cota inferior supera la mejor solución o cuando el estado de batería en un nodo hace infactible alcanzar una estación de recarga o la base.

```
BRANCH_AND_BOUND(M, n):
    // M = matriz de costos (n+1) x (n+1), vértice 0 = depósito
    // clientes = 1..n
    // Cota superior inicial con una heurística voraz
    (mejor_costo, mejor_ruta) ← NEAREST_NEIGHBOR(M, n)
    para v desde 0 hasta n:
        visitado[v] ← falso
```

```
    visitado[0] ← verdadero
    ruta ← [0]
    EXPLORAR(0, 1, 0)
    retornar mejor_costo, mejor_ruta
EXPLORAR(u, nivel, costo_actual):
    // u = último vértice visitado
    // nivel = cantidad de vértices visitados (incluye el depósito)
    // Caso base: ya se visitaron todos, cerrar el ciclo
    si nivel == n + 1:
        total ← costo_actual + M[u][0]
        si total < mejor_costo:
            mejor_costo ← total
            mejor_ruta ← ruta + [0]
        retornar
    // Poda por cota inferior
    si costo_actual + COTA_INFERIOR(u) >= mejor_costo:
        retornar
    // Ramificación: probar primero los vecinos más cercanos
    candidatos ← vértices v con visitado[v] = falso, ordenados por M[u][v]
    para cada v en candidatos:
        visitado[v] ← verdadero
        agregar v a ruta
        EXPLORAR(v, nivel + 1, costo_actual + M[u][v])
        quitar v de ruta                    // backtracking
        visitado[v] ← falso
COTA_INFERIOR(u):
    // Estimación optimista del costo restante: u → (no visitados) → 0
    pendientes ← vértices con visitado[v] = falso
    // Salida desde u hacia algún pendiente
    cota ← mín { M[u][v] : v en pendientes }
```

```
    // Salida desde cada pendiente hacia otro pendiente o al depósito
    para cada v en pendientes:
        cota ← cota + mín { M[v][w] : w en pendientes ∪ {0}, w ≠ v }
    retornar cota
```

- **Prim o Kruskal + DFS:** Permite evaluar el costo de conectar todas las estaciones de recarga con un árbol de expansión mínima comparando diferentes escenarios de inversión.

```
PRIM(M, N):
    // M = matriz de costos N x N (N = n + 1), vértice 0 = depósito
    para v desde 0 hasta N - 1:
        clave[v] ← ∞
        padre[v] ← NULO
        en_arbol[v] ← falso
    clave[0] ← 0
    repetir N veces:
        u ← vértice con en_arbol[u] = falso y clave[u] mínima
        en_arbol[u] ← verdadero
        para cada v desde 0 hasta N - 1:
            si no en_arbol[v] y M[u][v] < clave[v]:
                clave[v] ← M[u][v]
                padre[v] ← u
    // Construir lista de adyacencia del árbol
    para v desde 1 hasta N - 1:
        agregar v a hijos[padre[v]]
    retornar hijos


KRUSKAL(M, N):
    aristas ← lista de (M[i][j], i, j) para todo i < j
    ordenar aristas por costo ascendente
```

```
    para v desde 0 hasta N - 1:
        raiz[v] ← v
        rango[v] ← 0
    arbol ← lista vacía
    para cada (c, i, j) en aristas:
        si ENCONTRAR(i) ≠ ENCONTRAR(j):
            agregar (i, j) a arbol
            UNIR(i, j)
            si |arbol| == N - 1:
                romper
    retornar arbol
ENCONTRAR(x):
    si raiz[x] ≠ x:
        raiz[x] ← ENCONTRAR(raiz[x])     // compresión de caminos
    retornar raiz[x]
UNIR(a, b):
    ra ← ENCONTRAR(a)
    rb ← ENCONTRAR(b)
    si rango[ra] < rango[rb]:
        raiz[ra] ← rb
    sino:
        raiz[rb] ← ra
        si rango[ra] == rango[rb]:
            rango[ra] ← rango[ra] + 1
```

- **Knapsack:** Debido a que el dron tiene un presupuesto de energía, este algoritmo permitirá solucionar el problema de seleccionar el subconjunto de clientes que maximice el beneficio total.

```
        KNAPSACK_01(valor, peso, n, W):
    // valor[i], peso[i] para i = 1..n
```

```
// W = capacidad de la mochila
para i desde 0 hasta n:
    para c desde 0 hasta W:
        si i == 0 o c == 0:
            dp[i][c] ← 0
        sino si peso[i] <= c:
            dp[i][c] ← max( dp[i-1][c],
                            dp[i-1][c - peso[i]] + valor[i] )
        sino:
            dp[i][c] ← dp[i-1][c]
// Reconstrucción de los objetos elegidos
elegidos ← lista vacía
c ← W
para i desde n hasta 1:
    si dp[i][c] ≠ dp[i-1][c]:
        agregar i a elegidos
        c ← c - peso[i]
retornar dp[n][W], elegidos
```

- **Set Cover:** Permitirá identificar dónde colocar el centro de origen y las estaciones de recarga.

5. Lista formal de requerimientos, arquitectura y plan de pruebas

5.1 Requerimientos funcionales

Tabla 5. Requerimientos funcionales del sistema.

ID	Descripción
RF01	**Registro y carga de escenarios** El sistema deberá permitir cargar o registrar la base logística, clientes, estaciones de recarga, coordenadas, prioridades y parámetros generales de la misión.
RF02	**Registro de obstáculos y zonas restringidas**

	El sistema deberá permitir definir obstáculos y zonas de exclusión mediante polígonos o segmentos, indicando las regiones que una trayectoria no puede atravesar.
RF03	**Motor de geometría computacional** El sistema deberá implementar operaciones de orientación, producto cruz, intersección de segmentos, distancia, pertenencia de puntos a polígonos y otras operaciones geométricas justificadas por la solución.
RF04	**Validación de trayectorias** El sistema deberá determinar si un segmento de vuelo entre dos puntos es válido o intersecta una zona restringida, evitando construir aristas geométricamente imposibles.
RF05	**Construcción del grafo de navegación** El sistema deberá construir un grafo ponderado que represente conexiones válidas entre la base, clientes, estaciones y puntos auxiliares de navegación.
RF06	**Cálculo de caminos mínimos** El sistema deberá calcular la mejor trayectoria entre ubicaciones utilizando Dijkstra, Bellman-Ford, A* u otro algoritmo debidamente justificado según el escenario.
RF07	**Optimización exacta de misiones pequeñas** El sistema deberá implementar backtracking, branch and bound u otra estrategia exacta para determinar la secuencia óptima de visitas cuando el número de clientes permita su ejecución dentro del límite establecido.
RF08	**Optimización heurística de misiones grandes** El sistema deberá implementar al menos una estrategia greedy o heurística y una técnica de mejora local o aleatoria para producir soluciones de buena calidad en instancias donde la búsqueda exacta resulte inviable.
RF09	**Gestión de autonomía y estaciones de recarga** El sistema deberá verificar que la distancia o consumo acumulado entre puntos de recarga no supere la autonomía disponible y deberá incorporar recargas cuando sean necesarias.
RF10	**Búsqueda binaria sobre respuesta** El sistema deberá resolver al menos una variante paramétrica mediante búsqueda binaria, por ejemplo determinar la autonomía mínima necesaria para completar una misión bajo una función de factibilidad.
RF11	**Análisis estructural de la red** El sistema deberá utilizar DFS y, cuando corresponda, algoritmos de conectividad, biconectividad o detección de puntos críticos para analizar la robustez de la red de navegación.
RF12	**Diseño de red mínima de soporte**

	El sistema deberá permitir calcular un árbol de expansión mínima para conectar estaciones de recarga o nodos de soporte, comparando el costo de infraestructura de diferentes escenarios.
RF13	**Generación de datasets competitivos** El sistema deberá generar o importar conjuntos de prueba reproducibles de categorías Small, Medium, Large y Extreme, incluyendo casos aleatorios y casos adversariales.

5.2 Requerimientos no funcionales

Tabla 6. Requerimientos no funcionales del sistema.

ID	Descripción
RNF01	**Lenguaje y compilación** El sistema deberá desarrollarse en C++17 o superior y compilarse sin errores utilizando un compilador estándar compatible con el entorno definido por el equipo.
RNF02	**Modularidad** La arquitectura deberá ser modular, separando geometría, grafos, optimización, generación de datasets, benchmark y presentación de resultados.
RNF03	**Algoritmos propios** Los algoritmos principales deberán ser implementados por el equipo; no se permitirá sustituirlos por bibliotecas externas que resuelvan directamente caminos mínimos, TSP, geometría o aproximación.
RNF04	**Reproducibilidad** Las pruebas deberán ser reproducibles. Los generadores aleatorios deberán permitir fijar una semilla y registrar los parámetros utilizados para reconstruir cada escenario.
RNF05	**Complejidad documentada** Cada algoritmo deberá documentar su complejidad temporal y espacial, además de sus condiciones de aplicabilidad y límites prácticos.
RNF06	**Límites por categoría** El sistema deberá trabajar con restricciones explícitas de tiempo y memoria para las categorías Small, Medium, Large y Extreme.
RNF07	**Eficiencia de implementación** La implementación deberá priorizar estructuras eficientes de la STL, I/O rápido y prácticas de C++ propias de programación competitiva.

RNF08	**Visualización y extensibilidad** La visualización gráfica será opcional y no podrá reemplazar la validación algorítmica. El resultado oficial será producido por el motor C++. La solución deberá admitir ampliación futura con nuevos algoritmos y tipos de escenarios sin rediseñar completamente los módulos principales.

5.3 Arquitectura modular en C++ (separación en capas)

El sistema se organiza en cinco capas con dependencias en un solo sentido (cada capa solo usa las inferiores), como muestra la Figura 2. Esto permite cambiar un algoritmo de optimización sin tocar la geometría ni el juez, y probar cada capa por separado.

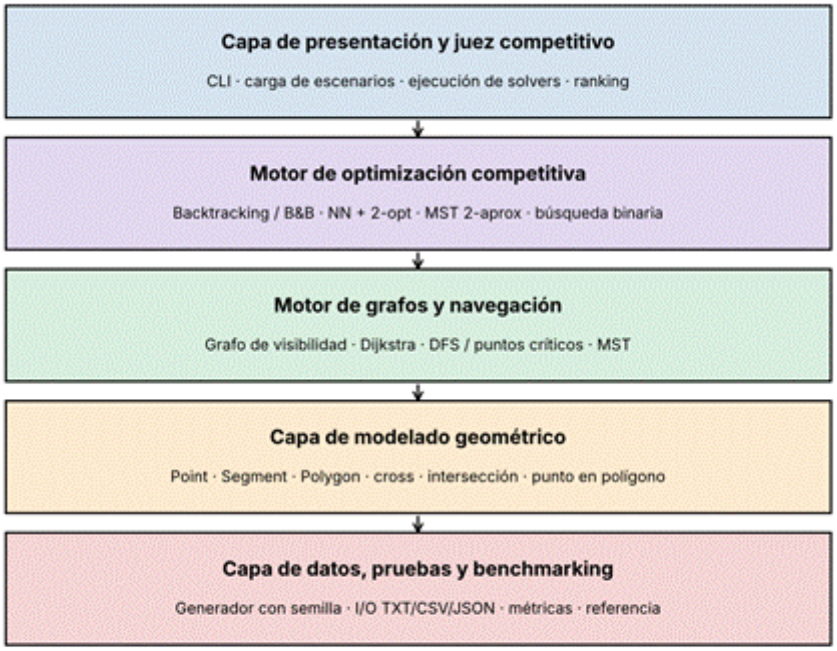


Figura 1. Arquitectura en cinco capas del sistema GeoDrone Challenge.

- **Capa de presentación y juez competitivo:** interfaz de consola para cargar escenarios, ejecutar solvers, visualizar métricas, validar resultados y comparar rankings por dataset.
- **Capa de modelado geométrico:** representa puntos, segmentos, polígonos y zonas restringidas; calcula orientación, intersecciones, distancias y pertenencia; valida movimientos y genera conexiones geométricamente factibles; provee el modelo al motor de grafos.

- **Motor de grafos y navegación:** construye el grafo de visibilidad, ejecuta caminos mínimos y DFS, y calcula conectividad, MST y métricas estructurales.
- **Motor de optimización competitiva:** ejecuta backtracking o branch and bound en instancias pequeñas, heurísticas, aleatorización y aproximación en instancias grandes, y búsqueda binaria sobre la respuesta; selecciona o compara estrategias según las restricciones del dataset.
- **Capa de datos, pruebas y benchmarking:** almacena escenarios, parámetros, semillas y soluciones de referencia; genera datasets Small, Medium, Large y Extreme; mide tiempo, memoria, factibilidad y calidad; exporta resultados y tablas comparativas.

La Tabla 7 asigna los módulos de código a cada capa. Se compilarán dos ejecutables con CMake: un solver y un judge/benchmark.

Tabla 7. Módulos C++ previstos, su capa y los requerimientos que cubren.

Módulo (archivo)	Responsabilidad	Capa	Requerimientos
geometry.hpp/.cpp	Point, Segment, Polygon, cross, intersección, punto en polígono, validez de segmento	Modelado geométrico	RF03, RF04
scenario.hpp, io.cpp	Estructuras del escenario; lectura y escritura TXT/CSV/JSON	Datos y benchmarking	RF01, RF02, RF15
visibility.cpp	Construcción del grafo de visibilidad	Grafos y navegación	RF05
shortest_path.cpp	Dijkstra y reconstrucción de rutas	Grafos y navegación	RF06
graph_analysis.cpp	DFS, puntos críticos, MST	Grafos y navegación	RF11, RF12
solver_exact.cpp	Backtracking / branch and bound	Optimización	RF07, RF09
solver_heur.cpp	Vecino más cercano, 2-opt, ILS, MST 2-aprox.	Optimización	RF08, RF09

param_search.cpp	Búsqueda binaria de la autonomía mínima	Optimización	RF10
generator.cpp	Datasets con semilla: aleatorios y adversariales	Datos y benchmarking	RF13
judge.cpp, main_judge.cpp	Verificación de factibilidad, medición, veredictos, ranking	Presentación y juez	RF14, RF15
main_solver.cpp, CMakeLists.txt	Ejecutable del solver y compilación	Presentación y juez	RF01

5.4 Plan de pruebas

El plan combina pruebas unitarias de geometría, pruebas de integración del grafo y los solvers, y pruebas de rendimiento con el juez. Todas las pruebas con aleatoriedad usan semillas fijas y registradas.

Tabla 8. Casos de prueba.

ID	Módulo	Entrada	Resultado esperado	Objetivo
PT01	Geometría	Ternas orientadas a izquierda, a derecha y colineales	cross > 0, < 0 y = 0	Orientación exacta
PT02	Geometría	Segmentos con cruce propio, en T, tocándose en un extremo, colineales solapados y colineales disjuntos	VERDADERO ×4 y FALSO	Intersección en bordes difíciles
PT03	Geometría	Puntos interior, exterior, sobre vértice, sobre borde y alineado con un vértice	INTERIOR, EXTERIOR, FRONTERA, FRONTERA, sin doble conteo	Punto en polígono
PT04	Validación	Vuelo que solo roza un vértice de un edificio	Válido	Regla de contacto
PT05	Validación	Vuelo a lo largo de una arista del edificio	Válido	Regla de colinealidad

PT06	Validación	Vuelo colineal que entra al interior de un polígono cóncavo / cruza una diagonal	Inválido	Prueba del punto medio
PT07	Validación	Cliente en el interior de un obstáculo	Escenario marcado inválido; cliente inalcanzable	Entradas inválidas
PT08	Grafo	Escenario de la Figura 1	16 aristas de 28 posibles; B–C1 ausente	Construcción del grafo
PT09	Caminos mínimos	Escenario de la Figura 1; grafos aleatorios con $n \leq 100$	d(B, C1) = 11,211 ($\pm 10^{-3}$); Dijkstra = Floyd–Warshall	Corrección de Dijkstra
PT10	Grafo	Cliente encerrado por polígonos (grafo desconexo)	DFS detecta la componente; el solver responde INFEASIBLE sin colgarse	Conectividad
PT11	Autonomía	A menor que el viaje de ida y vuelta al cliente más lejano, sin estaciones	INFEASIBLE	Restricción R1
PT12	Búsqueda binaria	A igual a la distancia mínima exacta; $A - \varepsilon$	Factible; infactible; la búsqueda converge a A ($\pm\varepsilon$)	Monotonía y precisión
PT13	Exacto vs. heurístico	50 instancias Small con semillas fijas	Heurística nunca por debajo del óptimo; gap medio ≤ 5 %	Calidad de la heurística
PT14	Generador	Misma semilla dos veces; semillas distintas	Archivos idénticos (mismo hash); archivos distintos	Reproducibilidad
PT15	Ventanas	Obligatorio con l_i imposible; opcional con l_i imposible	INFEASIBLE; el opcional se omite y se penaliza	Restricción R5
PT16	Juez	Datasets Large y Extreme	AC dentro de T_{max} y M_{max}; TLE/MLE detectados	Rendimiento

PT17	Adversariales	Clientes sobre vértices y bordes; pasillos estrechos; clusters lejanos	Rutas válidas; sin fallos ni bucles	Robustez
PT18	Aproximación	Instancias Small de la relajación métrica	Costo del MST-aprox. ≤ 2 × óptimo	Garantía teórica
PT19	I/O	Ejemplo de la sección 3.2; entradas con campos faltantes	Lectura/escritura correctas; error claro en entradas mal formadas	Formato de archivos
PT20	DFS	Grafo con puntos de articulación conocidos	Se detectan exactamente esos puntos	Análisis estructural

5.5 Matriz de trazabilidad

Tabla 9. Trazabilidad entre requerimientos, módulos y pruebas.

Requerimiento	Módulo	Pruebas
RF01, RF02	scenario.hpp, io.cpp	PT07, PT19
RF03, RF04	geometry.cpp	PT01 – PT06
RF05	visibility.cpp	PT08, PT10
RF06	shortest_path.cpp	PT09
RF07	solver_exact.cpp	PT13
RF08	solver_heur.cpp	PT13, PT17, PT18
RF09	solver_exact.cpp, solver_heur.cpp	PT11, PT15
RF10	param_search.cpp	PT12
RF11	graph_analysis.cpp	PT10, PT20
RF12	graph_analysis.cpp	PT18
RF13	generator.cpp	PT14, PT17
RF14	judge.cpp	PT16
RF15	io.cpp, judge.cpp	PT19
RNF03, RNF05	Todos los módulos algorítmicos	Revisión de código y sección 4

RNF04, RNF06	generator.cpp, judge.cpp	PT14, PT16